

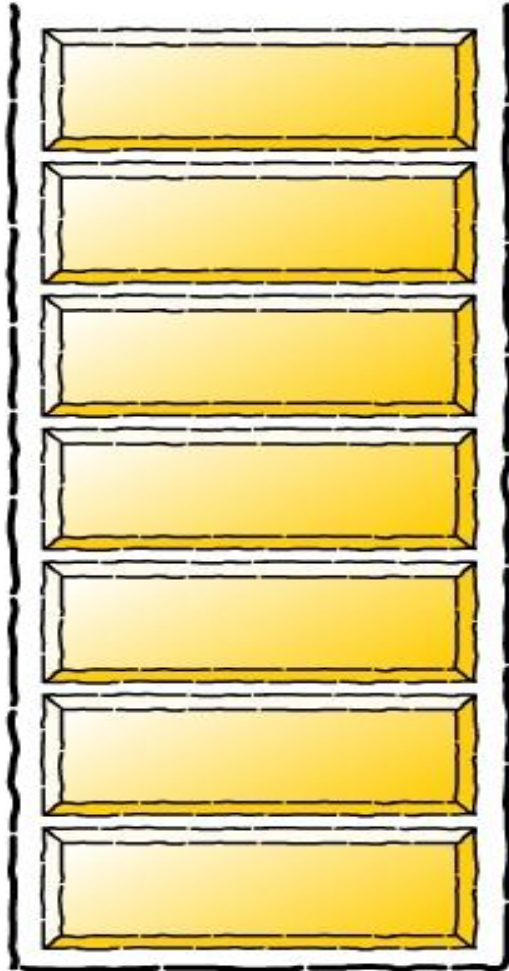
Data Structures

CSE207

Md. Manowarul Islam
Associate Professor
Dept. of CSE, Jagannath University

Infix to postfix conversion

Stack



Infix Expression

$(a + b - c) * d - (e + f)$

Postfix Expression

Postfix calculation

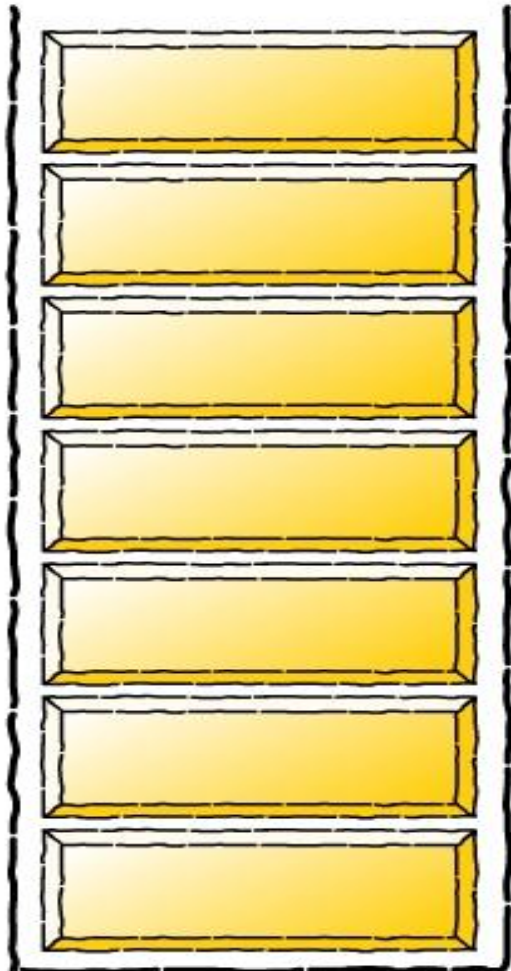
Scan the postfix expression left to right

- If the character x is a digit
 - Push it into the stack
- if the *character* is an operator (+, -, *, /)
 - pop two values from the stack
 - Perform the operation
 - Push the result of the operation into the stack

When all characters in postfix expression are processed pop the value from the stack and output it.

Postfix calculation

Stack



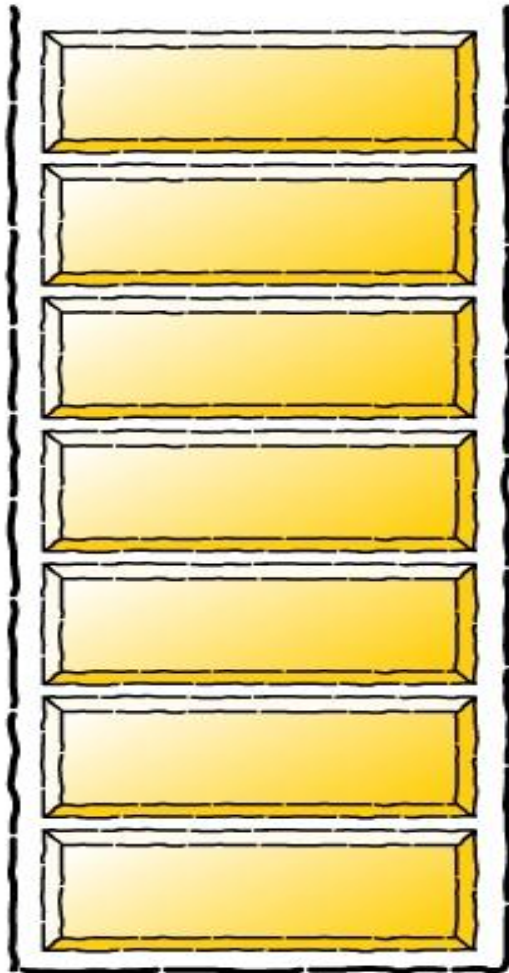
Postfix Expression

6 5 2 3 + 8 * + 3 + *

€ € € = €

Postfix calculation

Stack



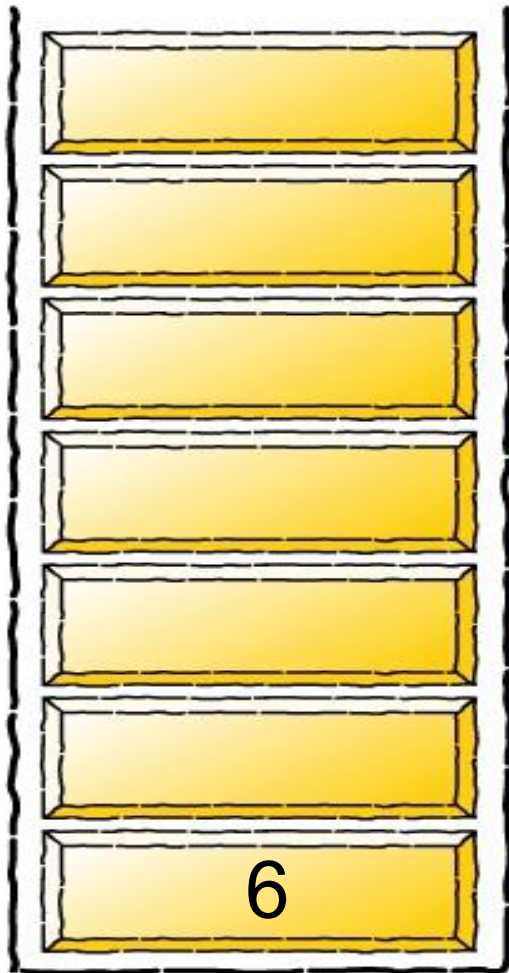
Postfix Expression

6 5 2 3 + 8 * + 3 + *)

€ € € = €

Postfix calculation

Stack



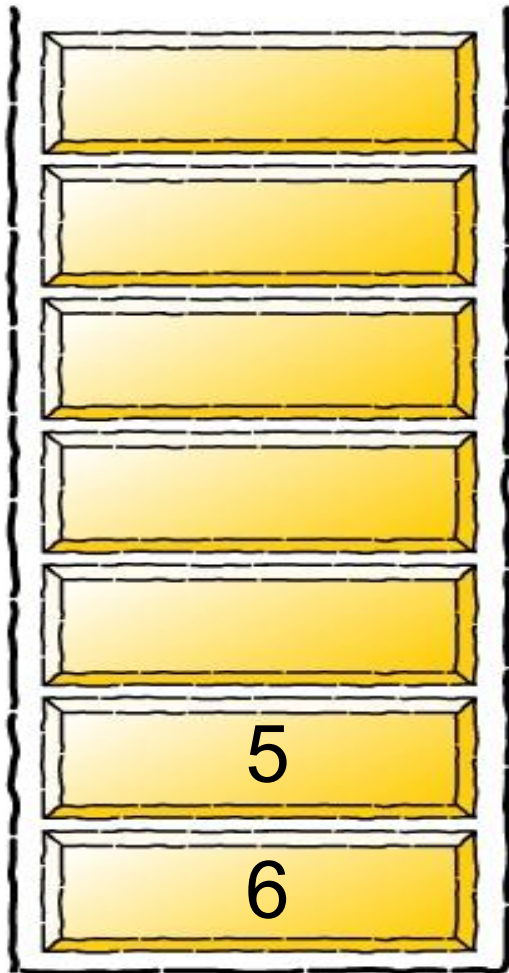
Postfix Expression

5 2 3 + 8 * + 3 + *)

€ € € = €

Postfix calculation

Stack



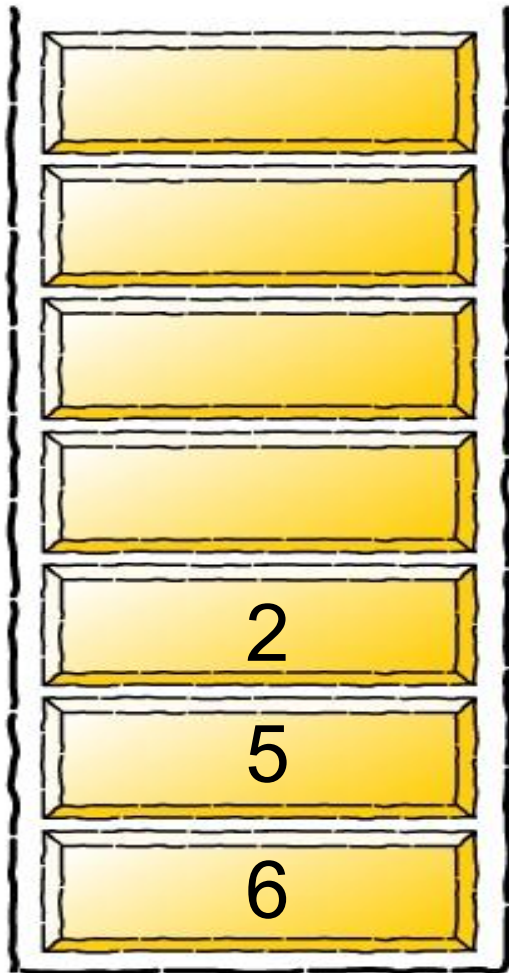
Postfix Expression

2 3 + 8 * + 3 + *)



Postfix calculation

Stack



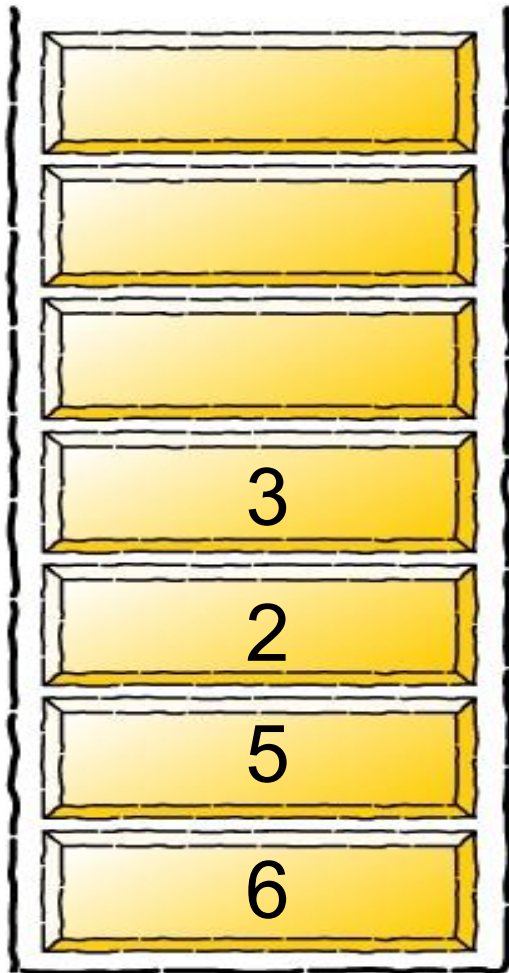
Postfix Expression

3 + 8 * + 3 + *)

€ € € = €

Postfix calculation

Stack



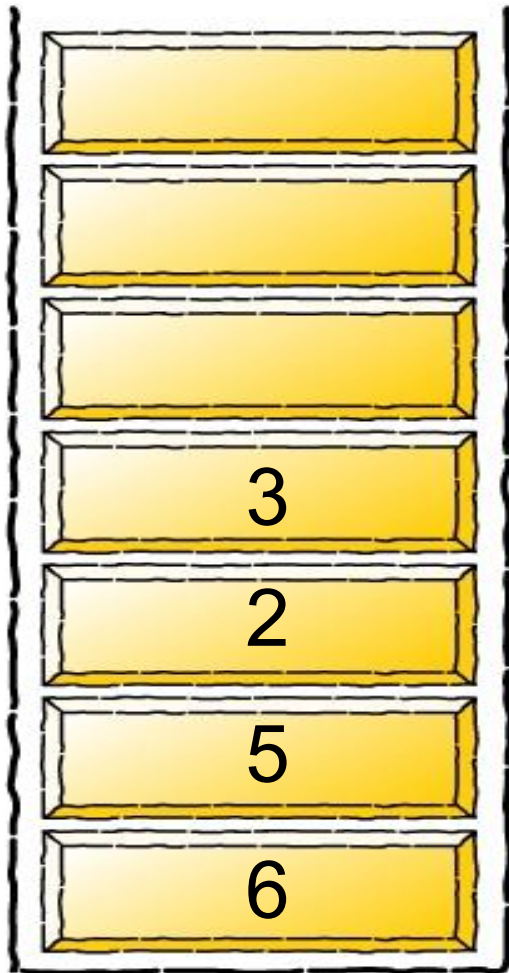
Postfix Expression

+ 8 * + 3 + *)

€ € € = €

Postfix calculation

Stack



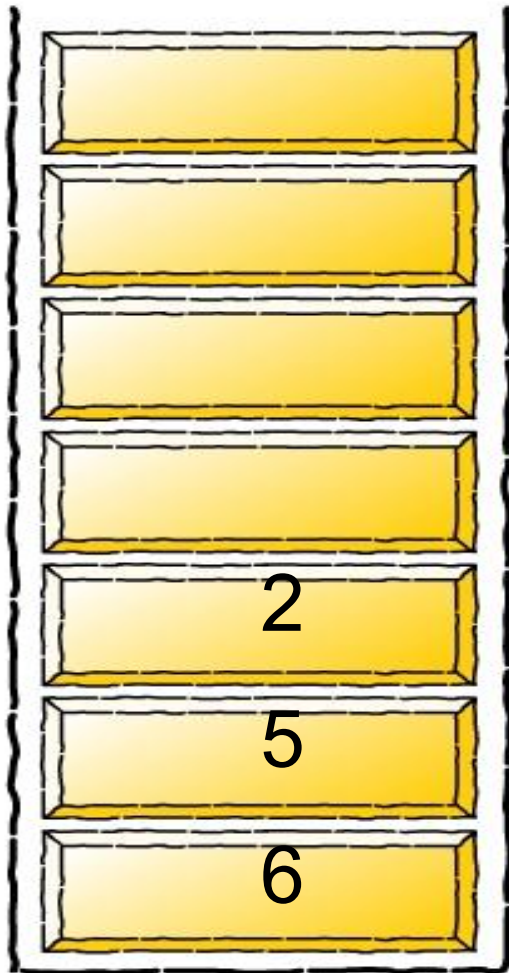
Postfix Expression

$8 * + 3 + *)$

$€ + € = €$

Postfix calculation

Stack



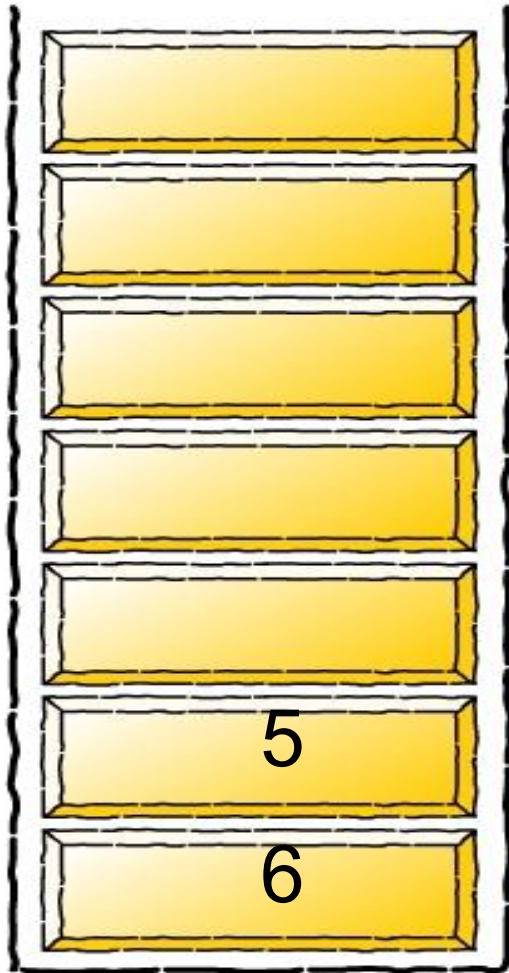
Postfix Expression

$8 * + 3 + *)$

$€ + 3 = €$

Postfix calculation

Stack



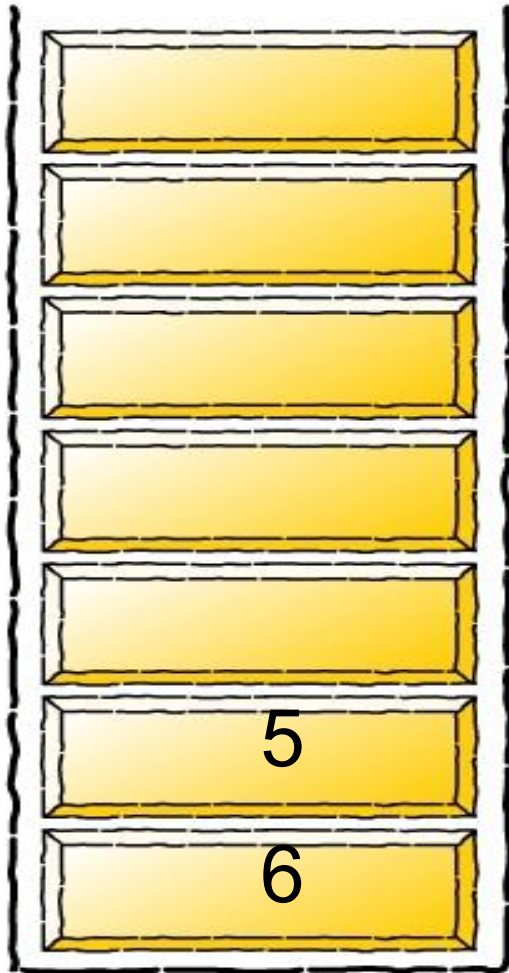
Postfix Expression

$8 * + 3 + *)$

$2 + 3 = €$

Postfix calculations

Stack



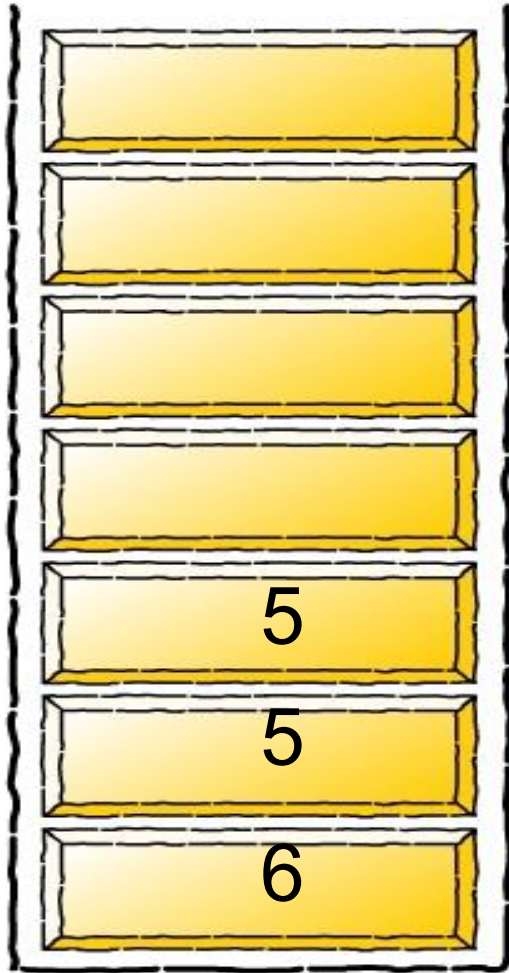
Postfix Expression

$8 * + 3 + *)$

$2 + 3 = 5$

Postfix calculations

Stack



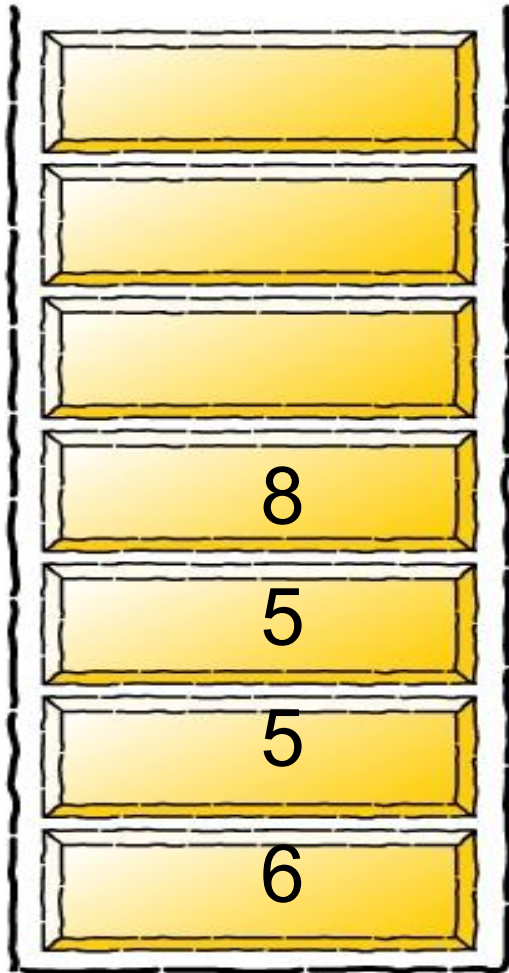
Postfix Expression

$8 * + 3 + *)$

$\text{€ € €} = \text{€}$

Postfix calculations

Stack



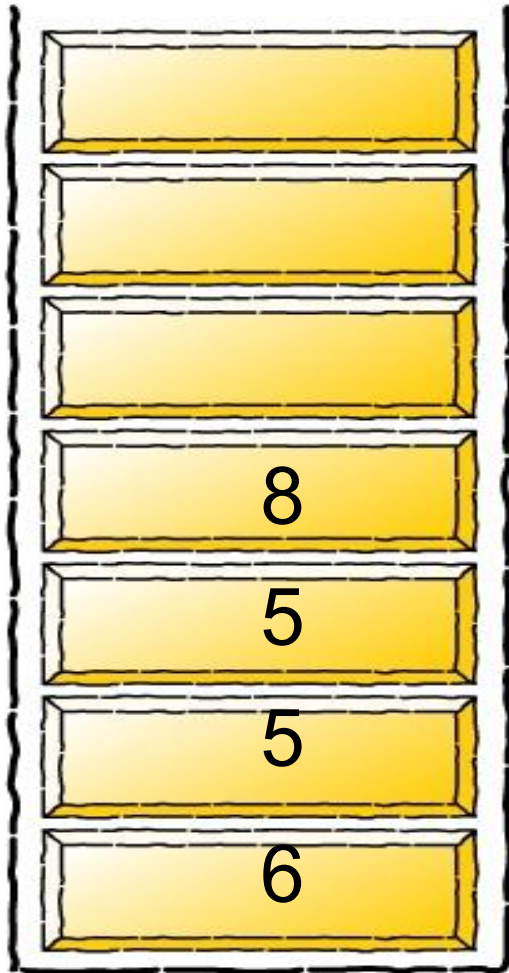
Postfix Expression

*** + 3 + *)**

€ € € = €

Postfix calculations

Stack



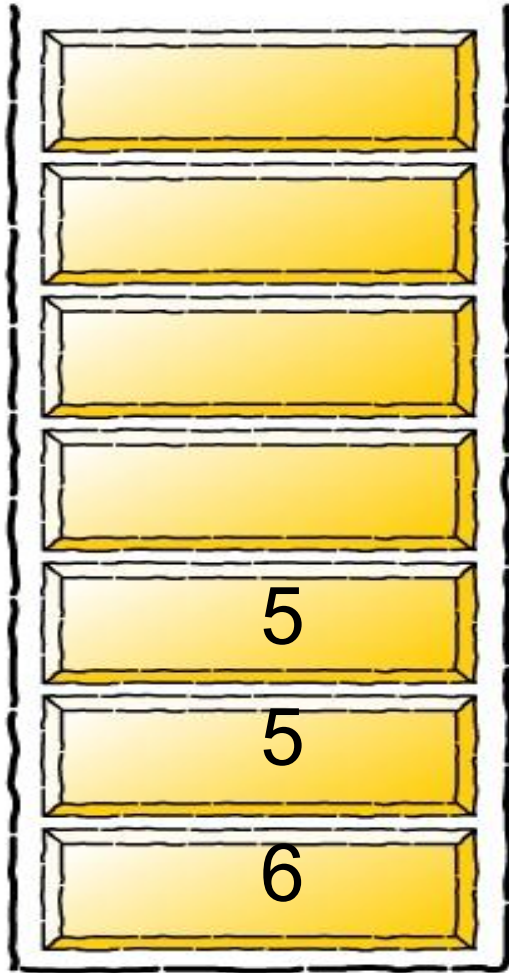
Postfix Expression

+ 3 + *)

€ * € = €

Postfix calculations

Stack



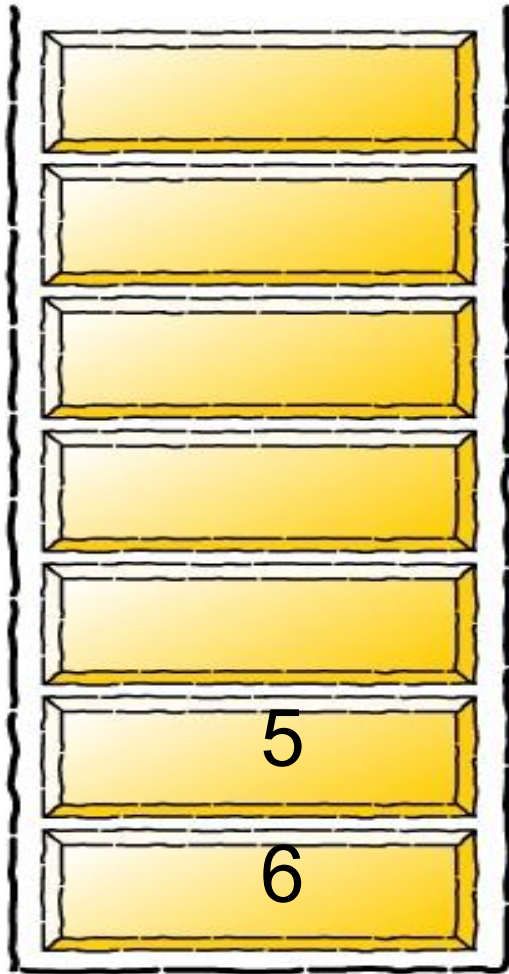
Postfix Expression

+ 3 + *)

€ * 8 = €

postfix calculations

Stack



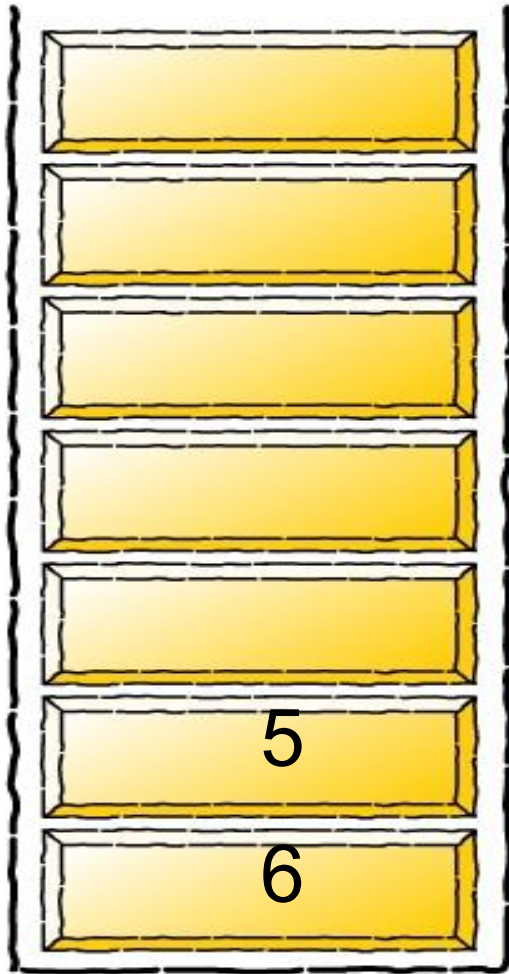
Postfix Expression

+ 3 + *)

5 * 8 = €

Postfix calculations

Stack



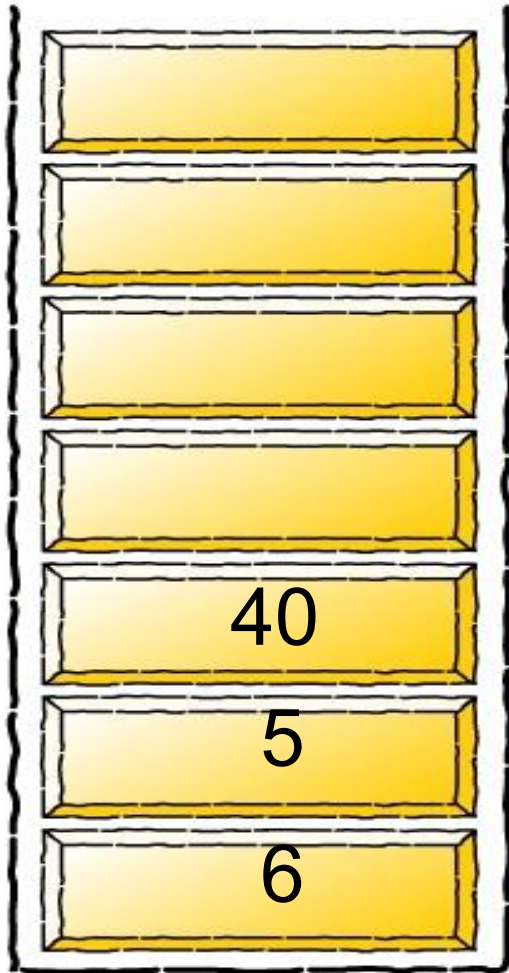
Postfix Expression

+ 3 + *)

5 * 8 = 40

Postfix calculations

Stack



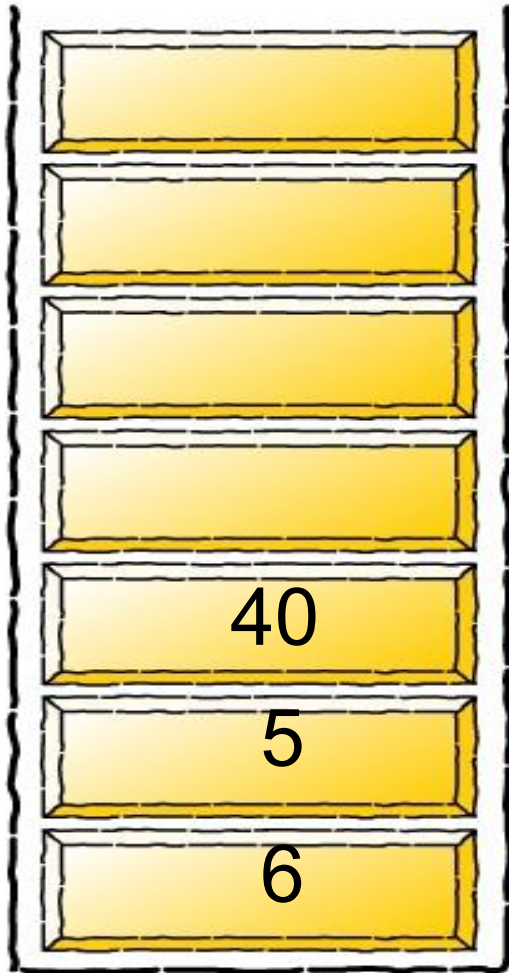
Postfix Expression

+ 3 + *)

€ € € = €

Postfix calculations

Stack



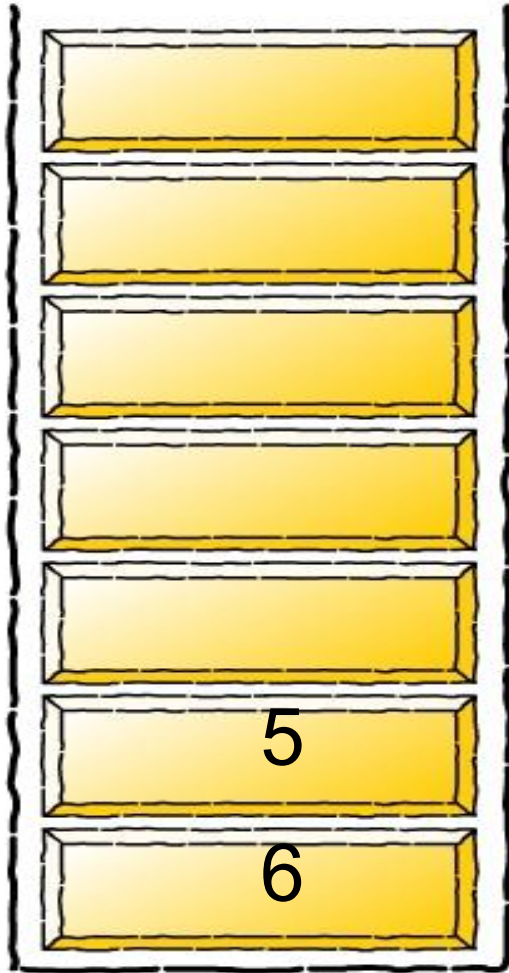
Postfix Expression

3 + *)

€ + € = €

Postfix calculations

Stack



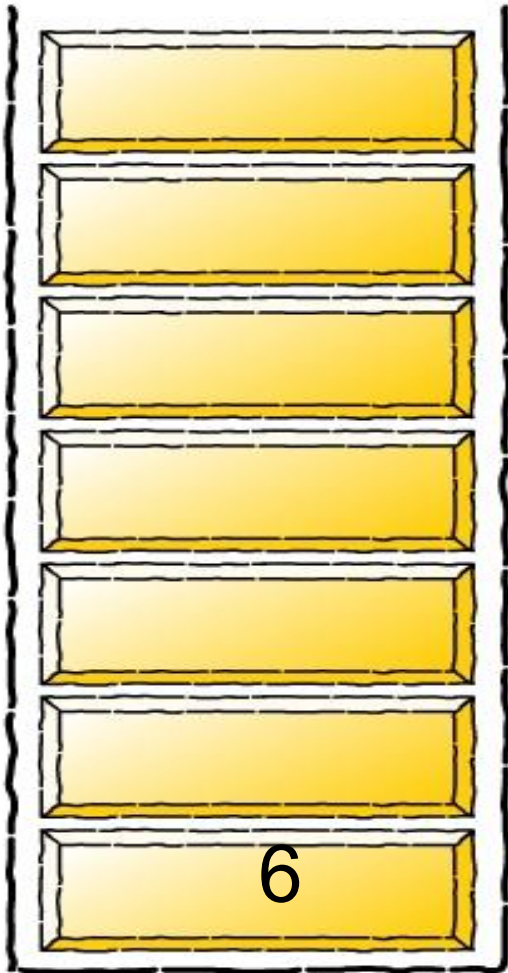
Postfix Expression

3 + *)

€ + 40 = €

Postfix calculations

Stack



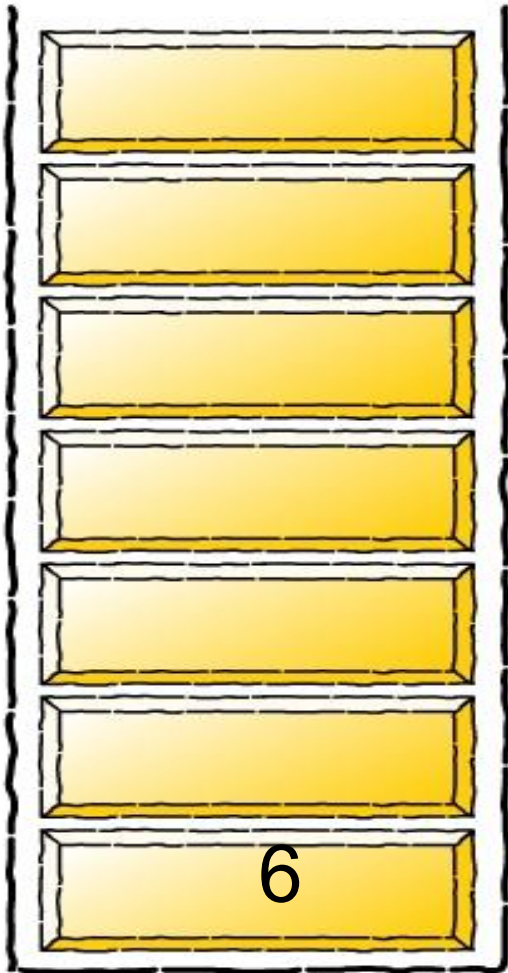
Postfix Expression

$3 + *)$

$5 + 40 = €$

Postfix calculations

Stack

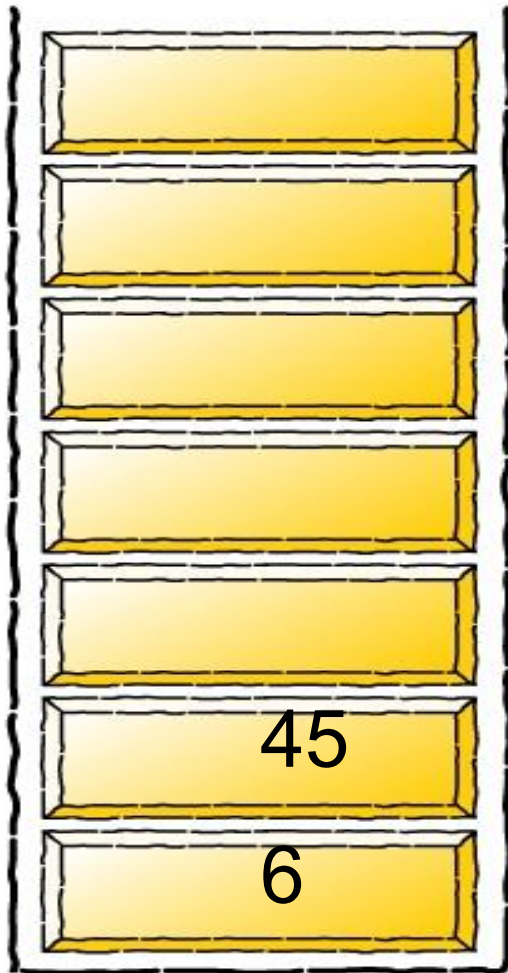


Postfix Expression

3 + *)

5 + 40 = 45

Stack



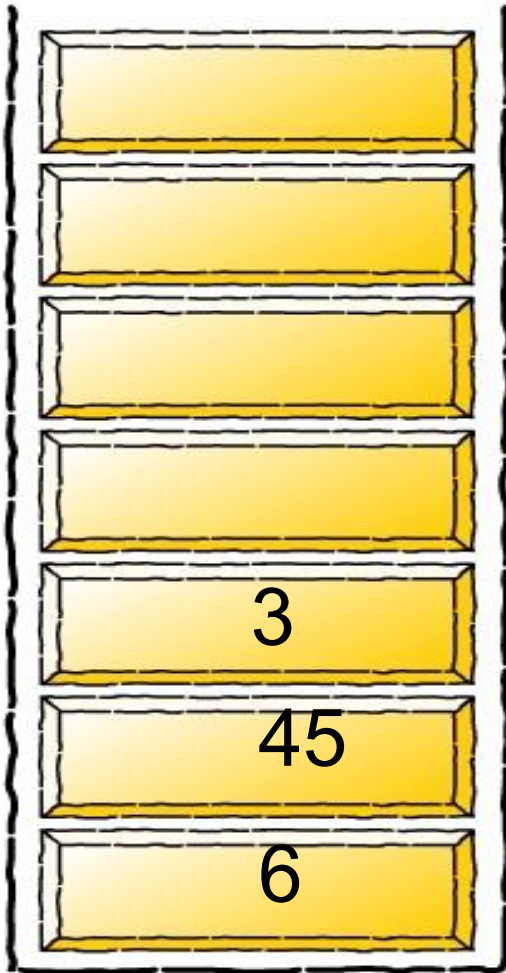
Postfix Expression

3 + *)

€ € € = €

Postfix calculations

Stack



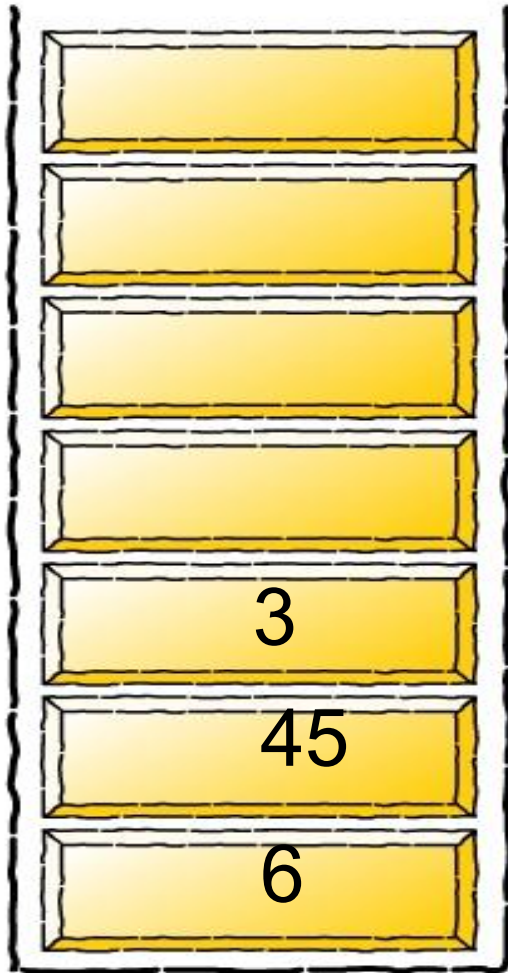
Postfix Expression

+ *)

€ € € = €

Postfix calculations

Stack



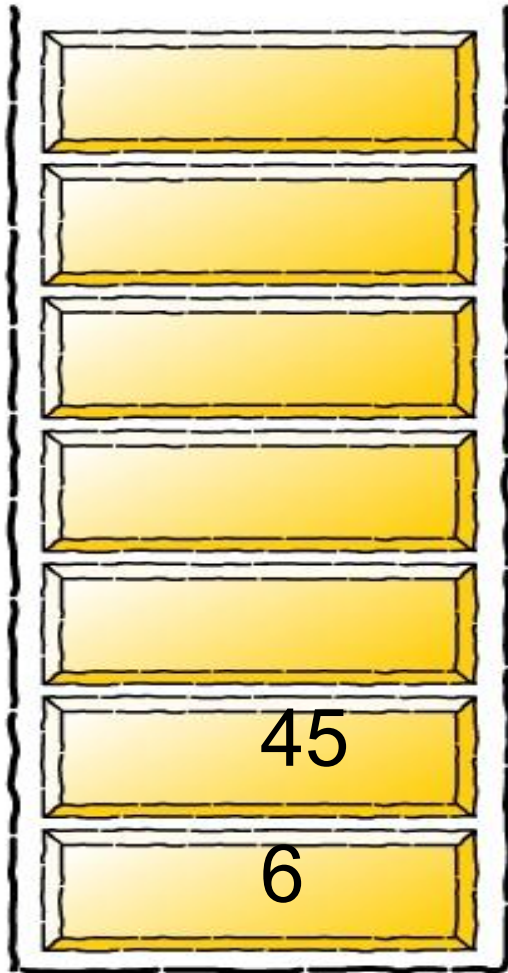
Postfix Expression

*)

€ + € = €

Postfix calculations

Stack



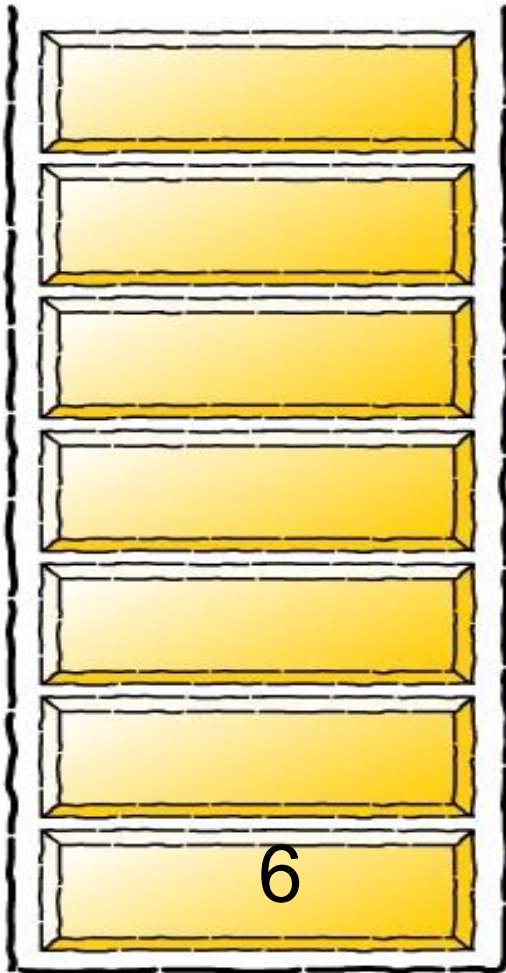
Postfix Expression

$\ast$)

$\text{€} + 3 = \text{€}$

Postfix calculations

Stack



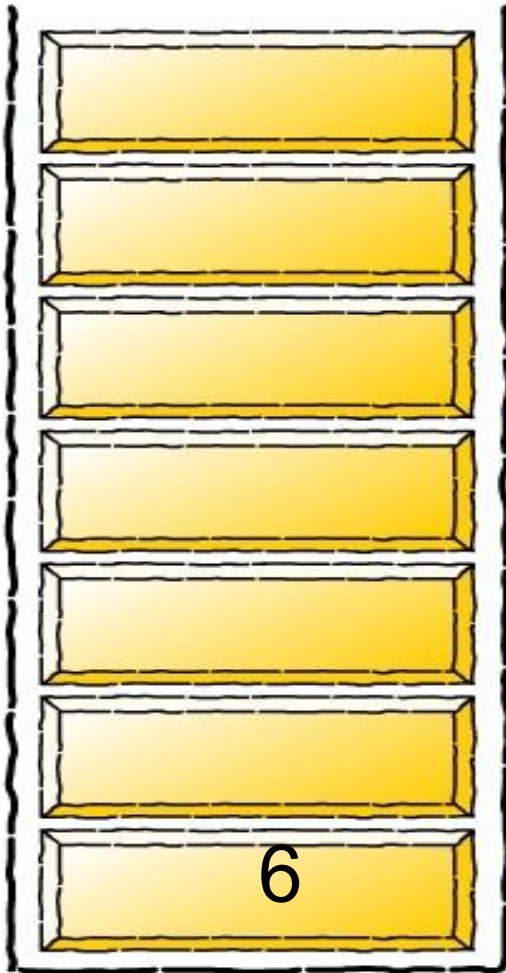
Postfix Expression

$\ast$)

$$45 + 3 = \text{€}$$

Postfix calculations

Stack



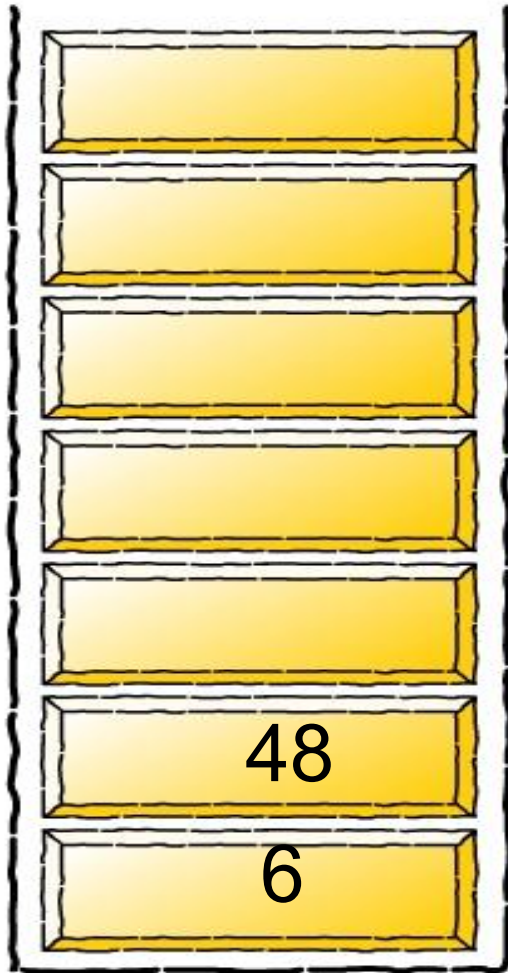
Postfix Expression

*)

$$45 + 3 = 48$$

Postfix calculations

Stack

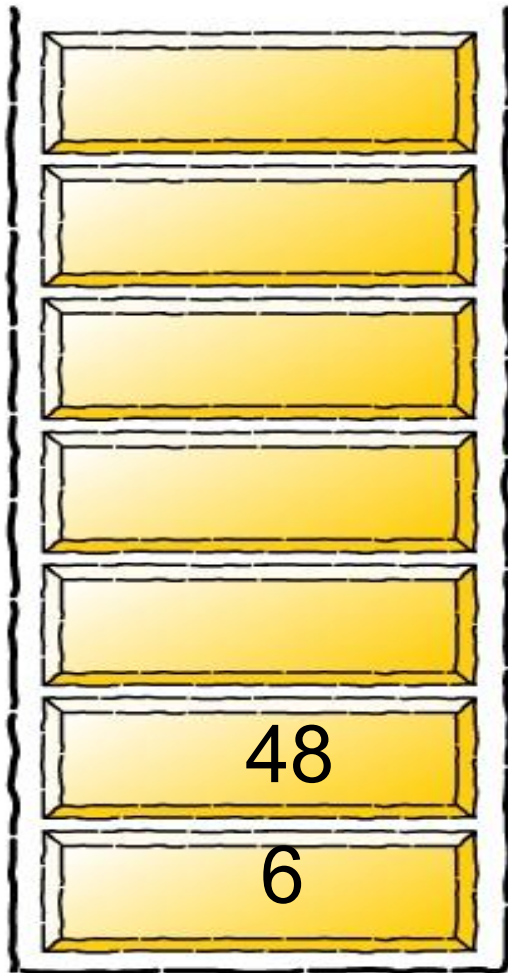


Postfix Expression

$\ast$)

$\text{€} \text{€} \text{€} = \text{€}$

Stack

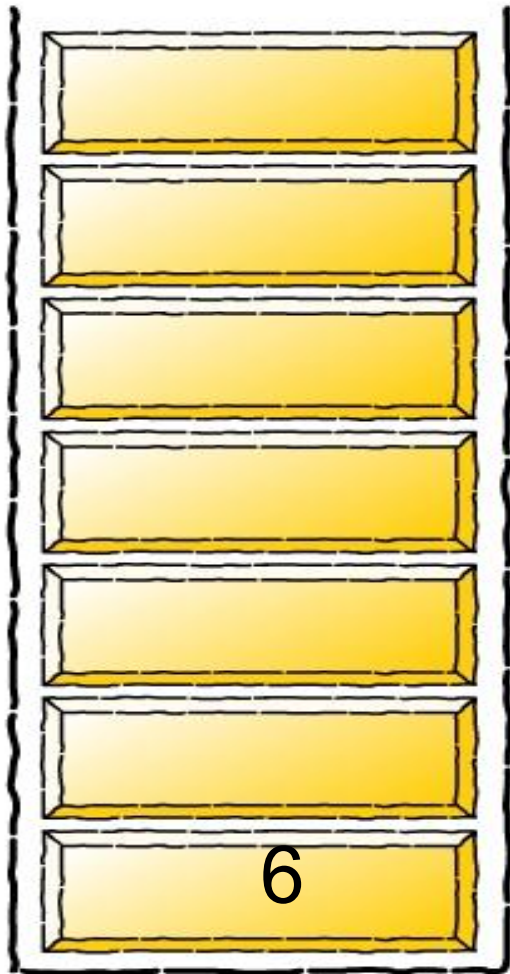


Postfix Expression

)

$$\text{€} * \text{€} = \text{€}$$

Stack



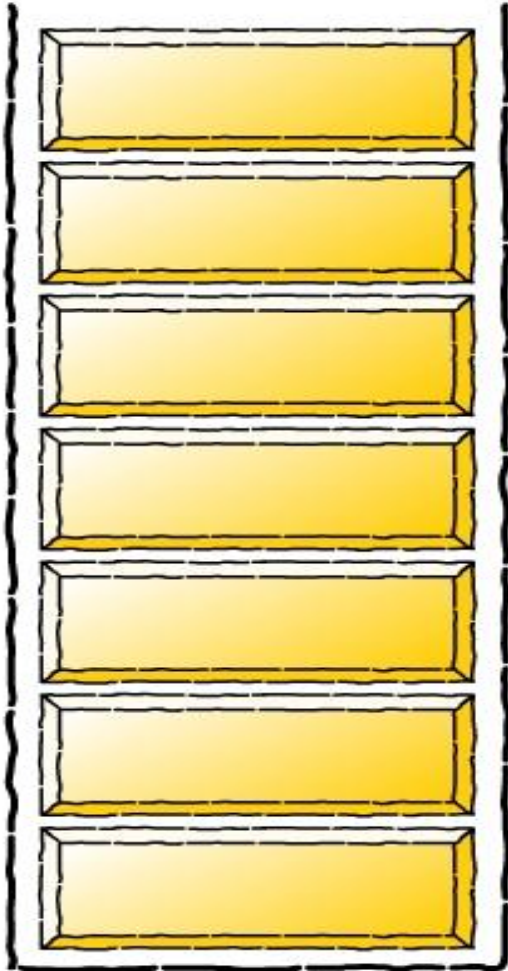
Postfix Expression

)

€ * 48 = €

Postfix calculations

Stack



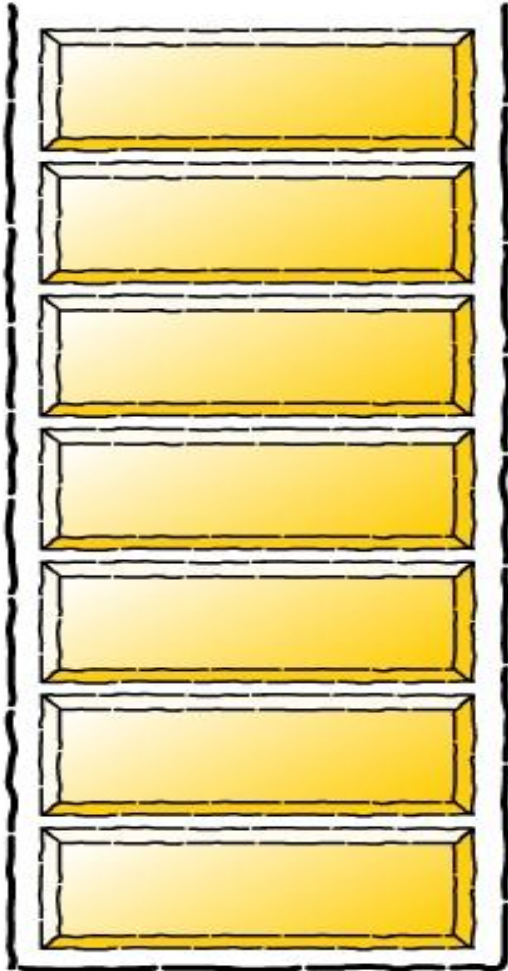
Postfix Expression

)

$6 * 48 = €$

Postfix calculations

Stack



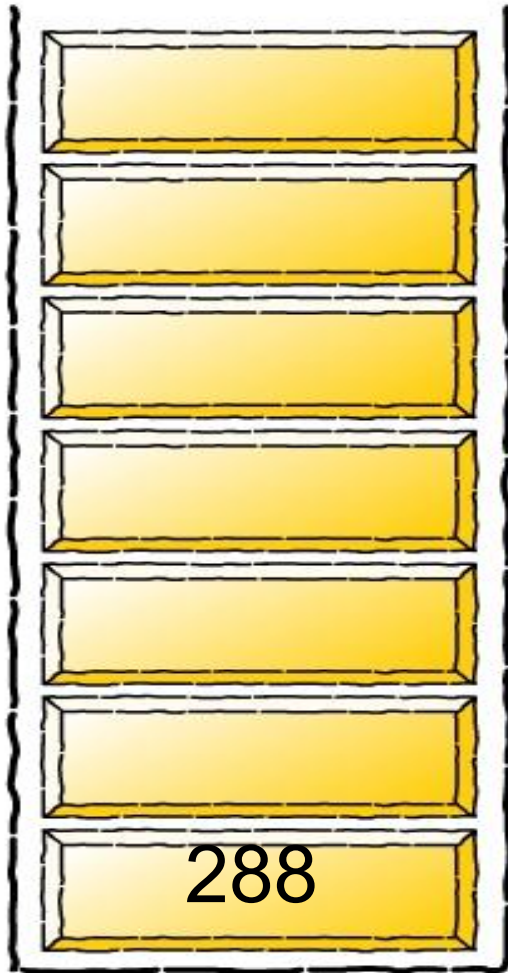
Postfix Expression

)

$$6 * 48 = 288$$

Postfix calculations

Stack



Postfix Expression



€ € € = €



Algorithm

Algorithm 6.5: This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

1. Add a right parenthesis “)” at the end of P. [This acts as a sentinel.]
2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel “)” is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator $\otimes$ is encountered, then:
 - (a) Remove the two top elements of STACK, where A is the top element and B is the next-to-top element.
 - (b) Evaluate $B \otimes A$.
 - (c) Place the result of (b) back on STACK.[End of If structure.]
- [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
6. Exit.

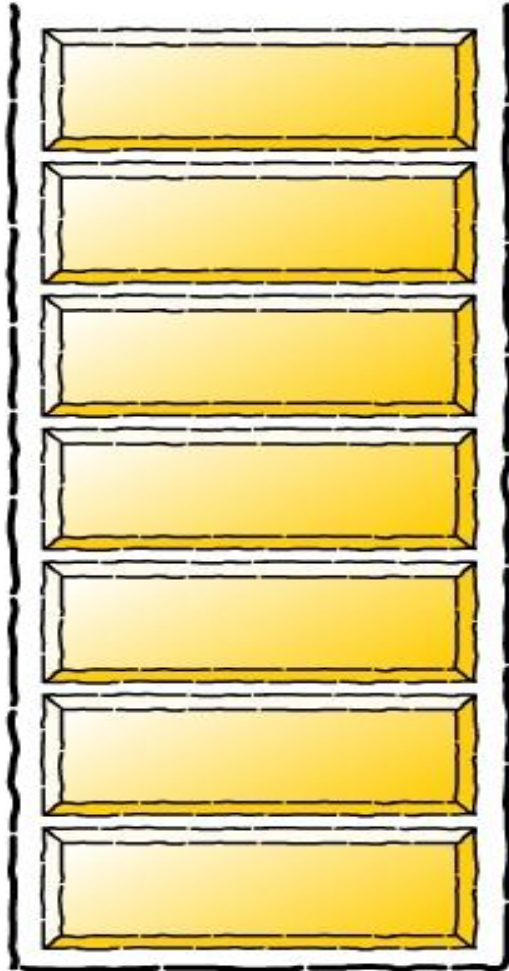
Example: P : 5, 6, 2, +, *, 12, 4, /, -) and Value: ?

Postfix Expression Q	Stack
5	5
6	5, 6
2	5, 6, 2
+	5, 8
*	40
12	40, 12
4	40, 12, 4
/	40, 3
-	37
)	37

Value: 37

Infix to postfix conversion

Stack



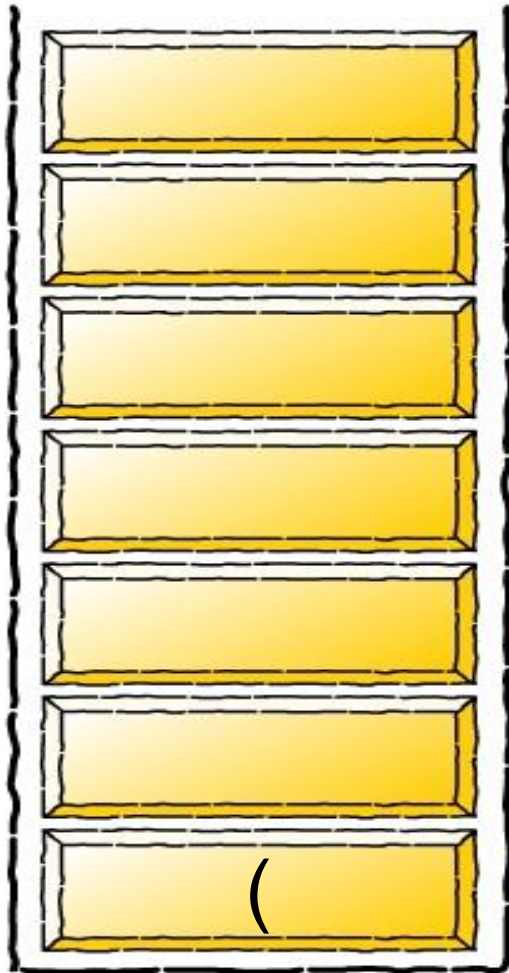
Infix Expression

$(a + b - c) * d - (e + f)$

Postfix Expression

Infix to postfix conversion

Stack



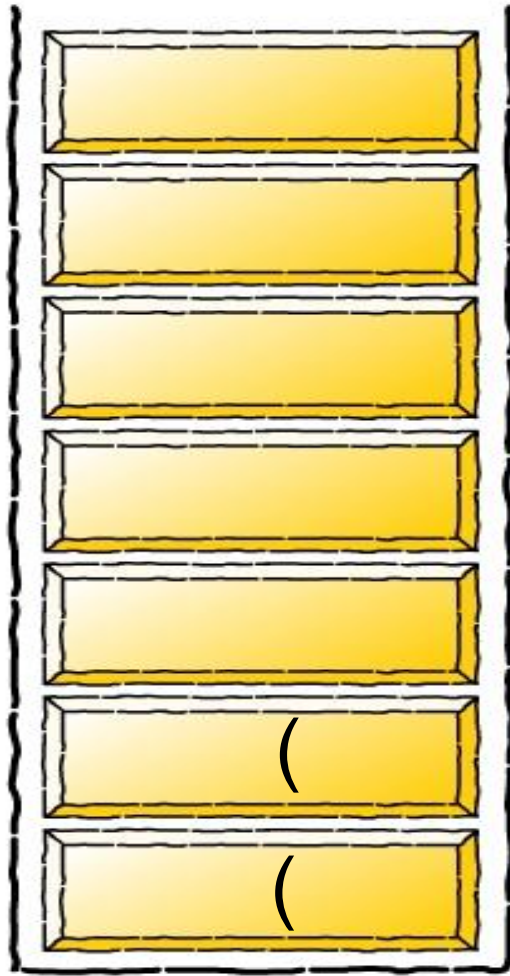
Infix Expression

$(a + b - c) * d - (e + f))$

Postfix Expression

Infix to postfix conversion

Stack



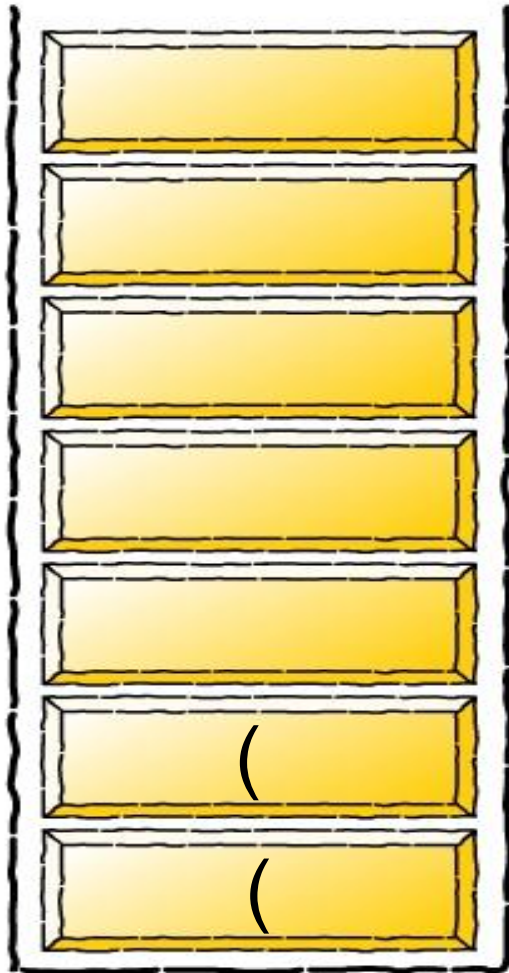
Infix Expression

$a + b - c) * d - (e + f))$

Postfix Expression

Infix to postfix conversion

Stack



Infix Expression

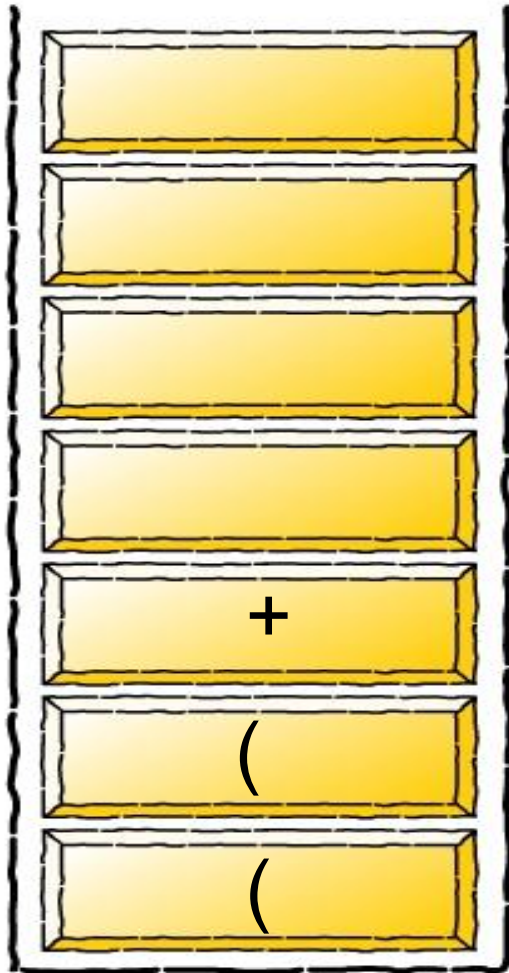
$+ b - c) * d - (e + f))$

Postfix Expression

a

Infix to postfix conversion

Stack



Infix Expression

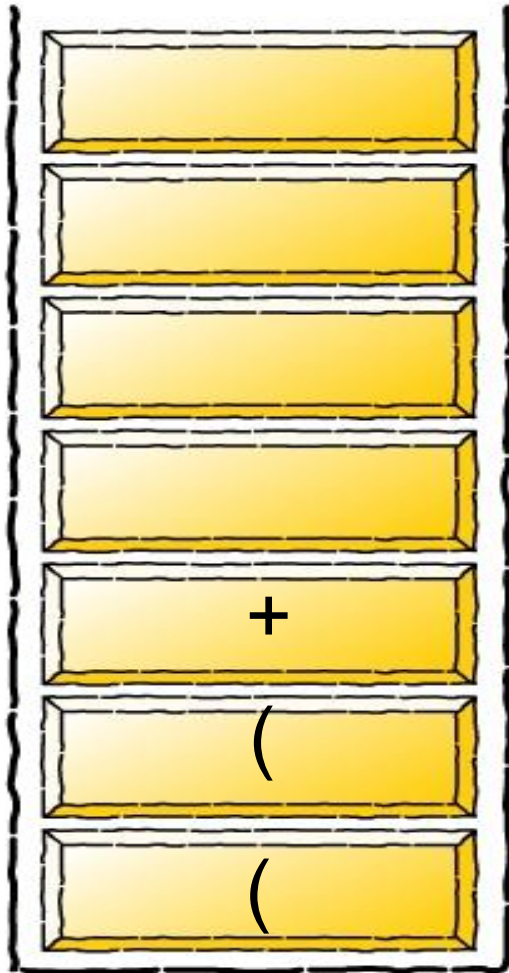
$b - c) * d - (e + f))$

Postfix Expression

a

Infix to postfix conversion

Stack



Infix Expression

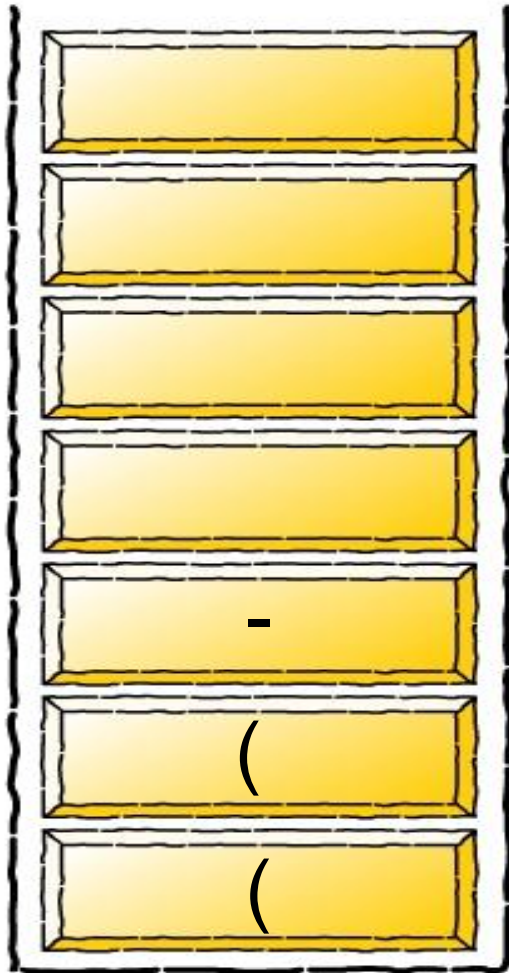
$- c) * d - (e + f))$

Postfix Expression

$a b$

Infix to postfix conversion

Stack



Infix Expression

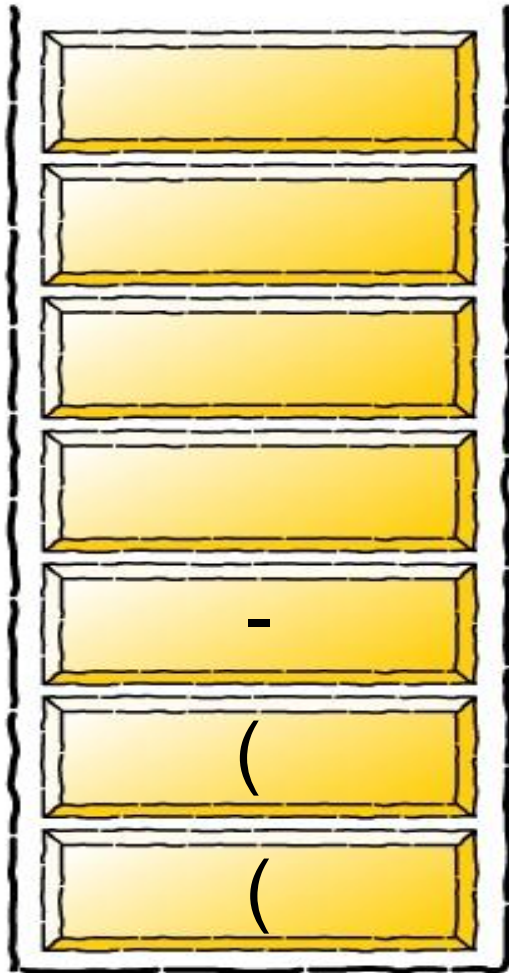
$c) * d - (e + f))$

Postfix Expression

$a b +$

Infix to postfix conversion

Stack



Infix Expression

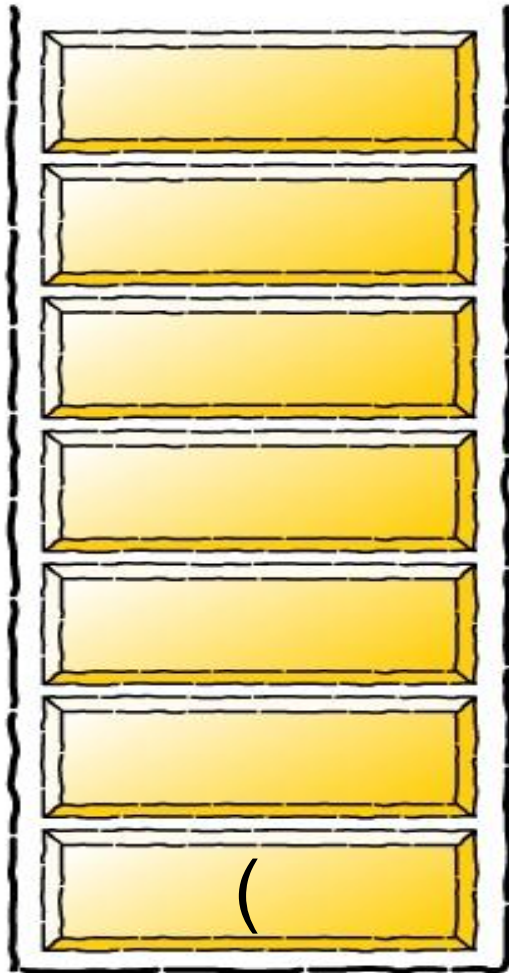
$) * d - (e + f))$

Postfix Expression

$a b + c$

Infix to postfix conversion

Stack



Infix Expression

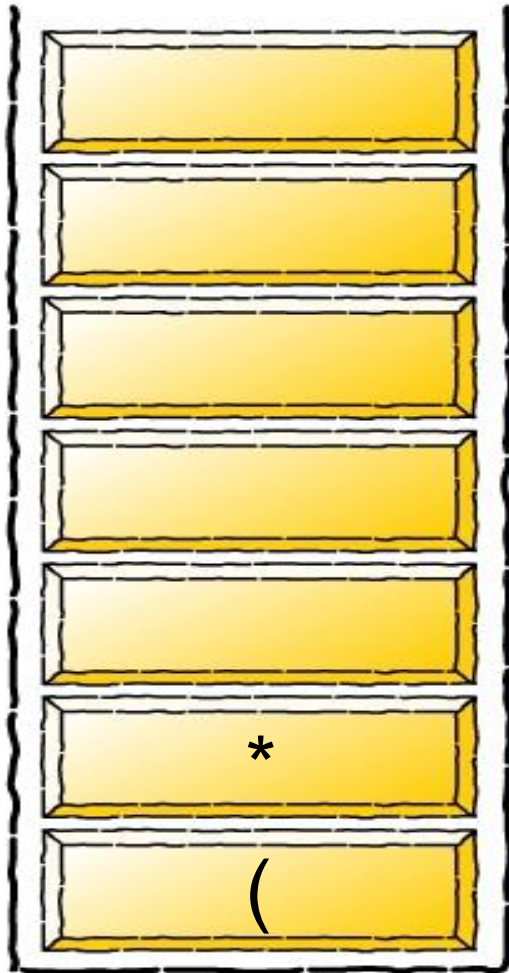
$* d - (e + f)$

Postfix Expression

$a b + c -$

Infix to postfix conversion

Stack



Infix Expression

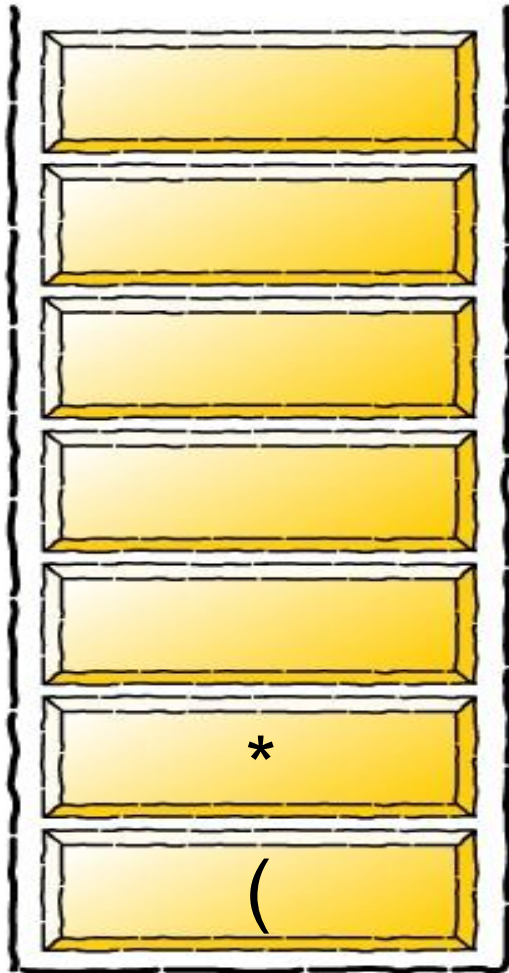
$d - (e + f)$

Postfix Expression

$a b + c -$

Infix to postfix conversion

Stack



Infix Expression

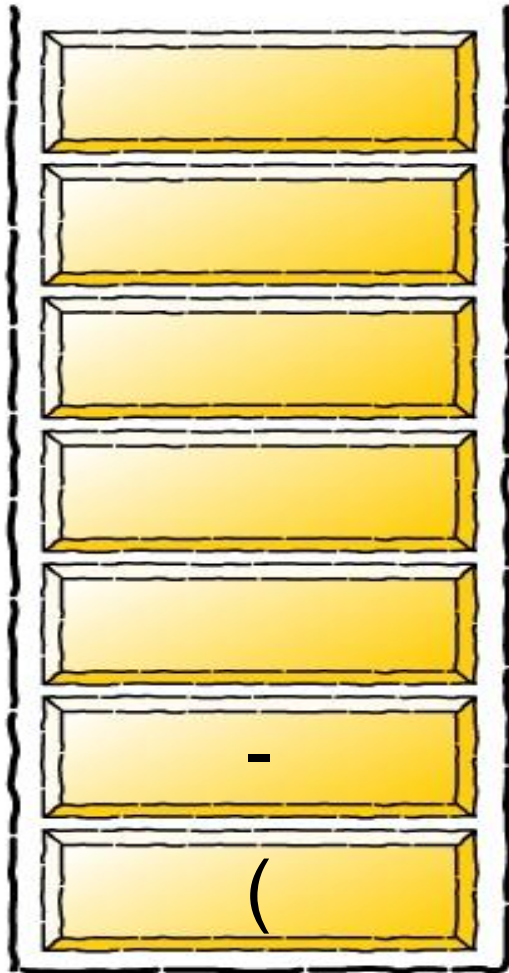
$- (e + f))$

Postfix Expression

$a b + c - d$

Infix to postfix conversion

Stack



Infix Expression

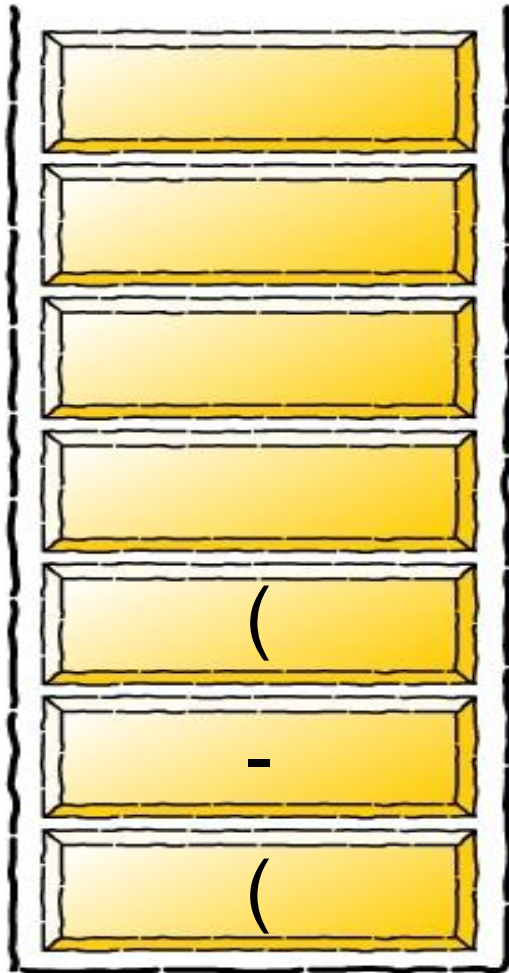
$(e + f))$

Postfix Expression

$a b + c - d *$

Infix to postfix conversion

Stack



Infix Expression

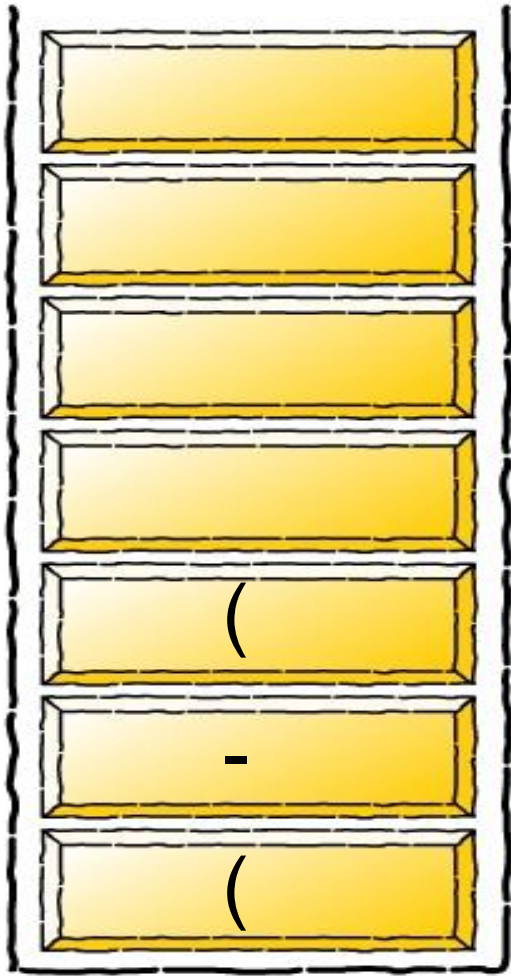
$e + f))$

Postfix Expression

$a b + c - d *$

Infix to postfix conversion

Stack



Infix Expression

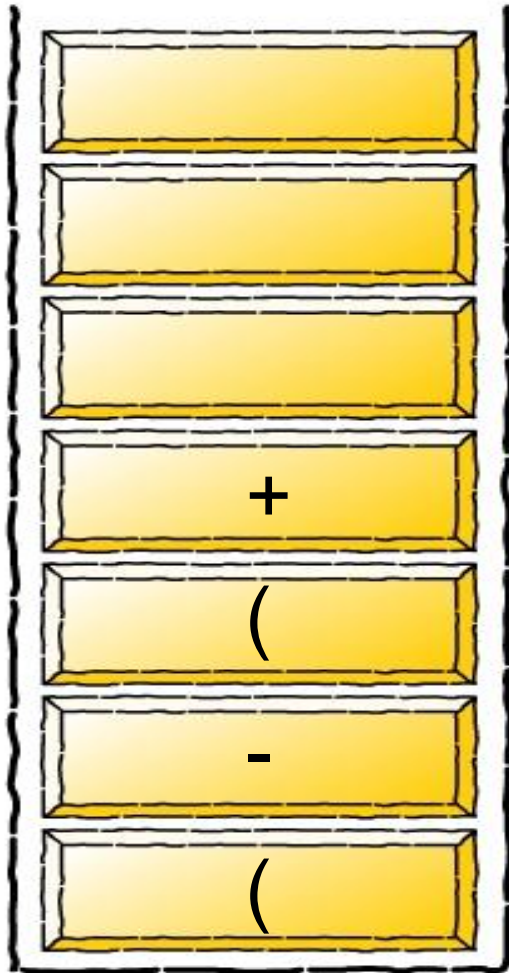
+ f))

Postfix Expression

a b + c - d * e

Infix to postfix conversion

Stack



Infix Expression

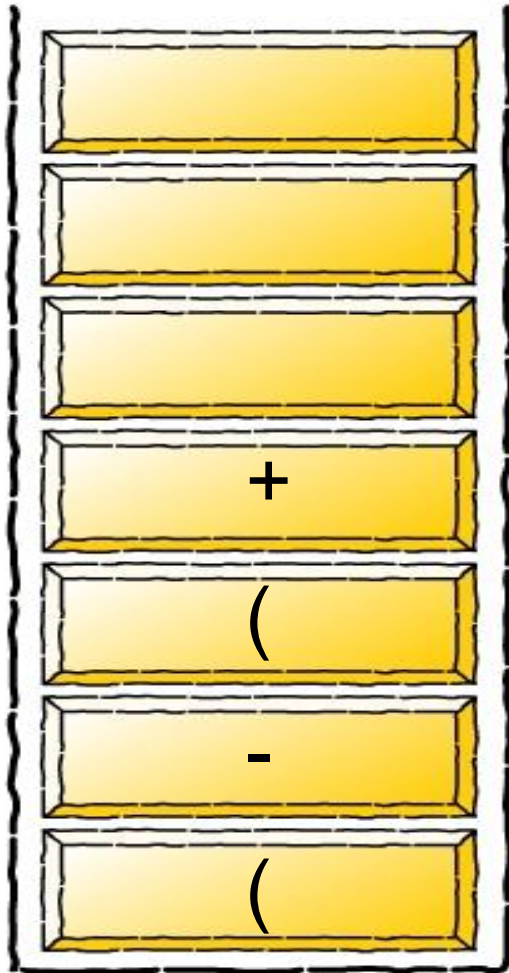
f))

Postfix Expression

a b + c - d * e

Infix to postfix conversion

Stack



Infix Expression

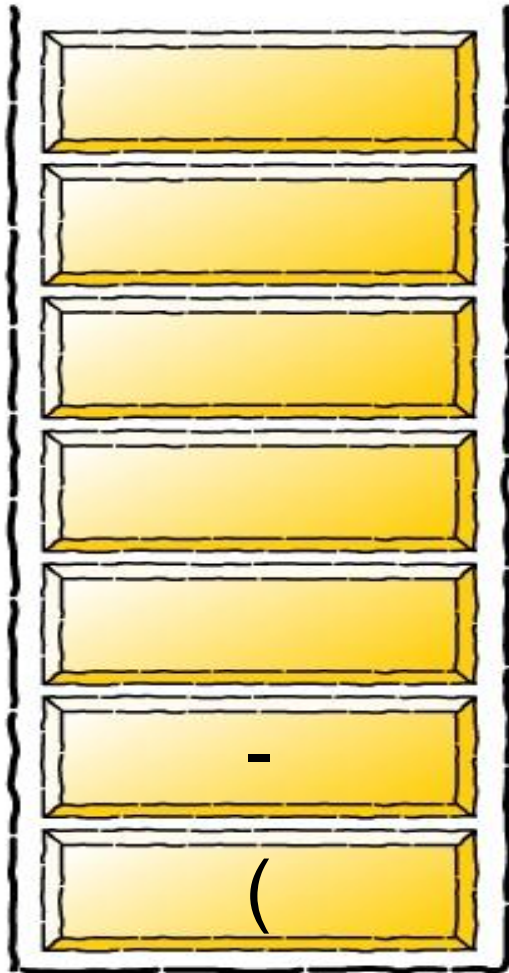
))

Postfix Expression

a b + c - d * e f

Infix to postfix conversion

Stack



Infix Expression

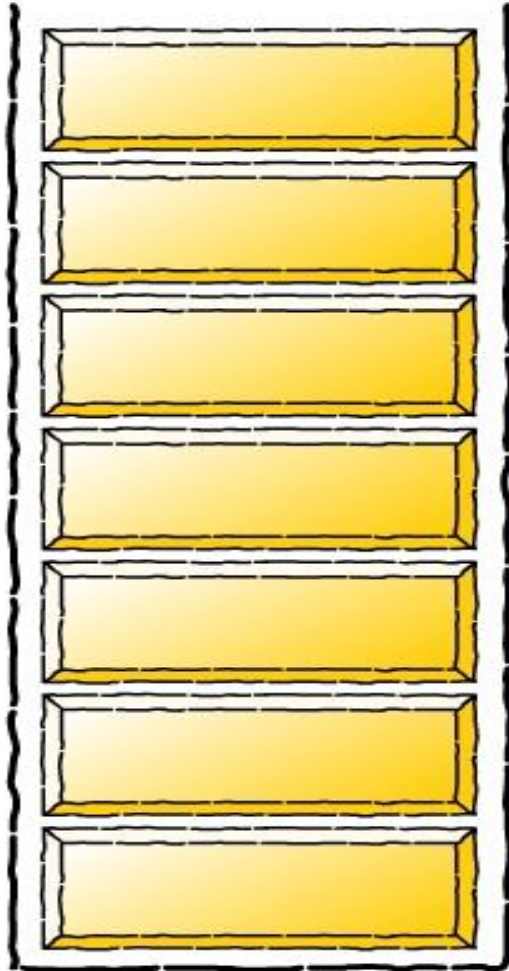
)

Postfix Expression

a b + c - d * e f +

Infix to postfix conversion

Stack



Infix Expression

Postfix Expression

$a b + c - d * e f + -$